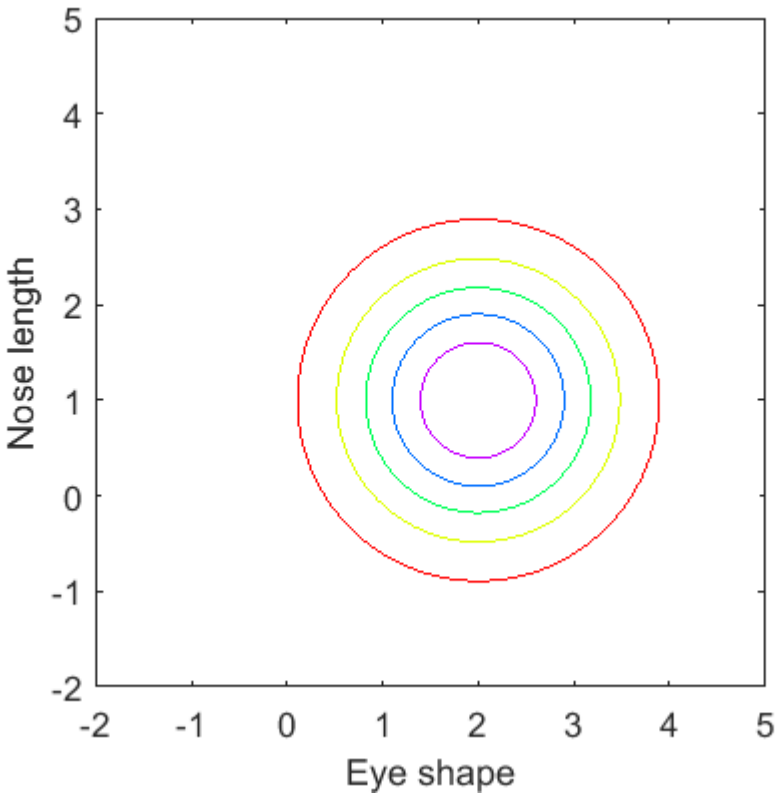
Czego uczy się macierz kowariancji w CMA-ES

Jarosław Arabas
Maksym Bieńkowski

Algorytm wspinaczkowy

$$generuj\ d_i(t) \sim N(0, I), i = 1 \ldots \lambda$$
$$oblicz\ q_i(t) = q\left(m(t) + \sigma(t) \cdot d_i(t)\right)$$
$$sortuj\ według\ q_i(t)$$
$$m(t+1) = m(t) + \sigma(t) \cdot d_1(t)$$

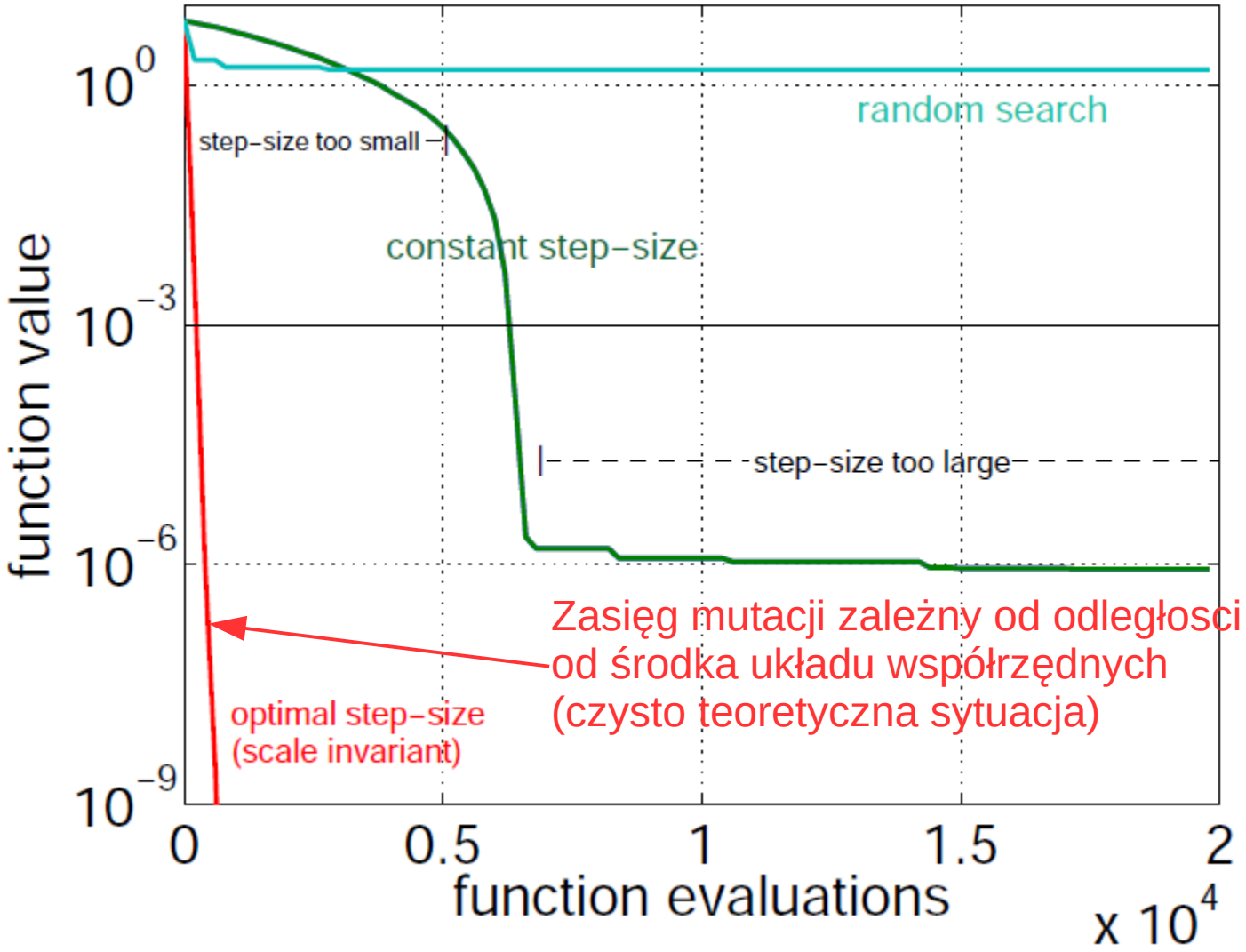
Algorytm wspinaczkowy



Poziomice
funkcji gęstości prawdopodobieństwa
standaryzowanego rozkładu normalnego
są okręgami

https://www.statisticshowto.com/bivariate-normal-distribution/

Zależność tempa zbieżności od zasięgu mutacji (alg. wspinaczkowy)

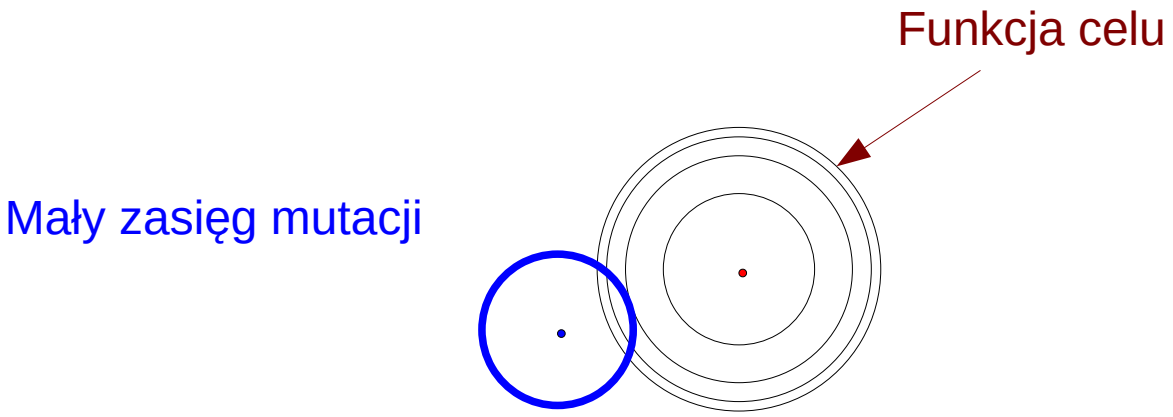


$$f(\boldsymbol{x}) = \sum_{i=1}^{n} x_i^2$$

in $[-0.2, 0.8]^n$
for $n = 10$

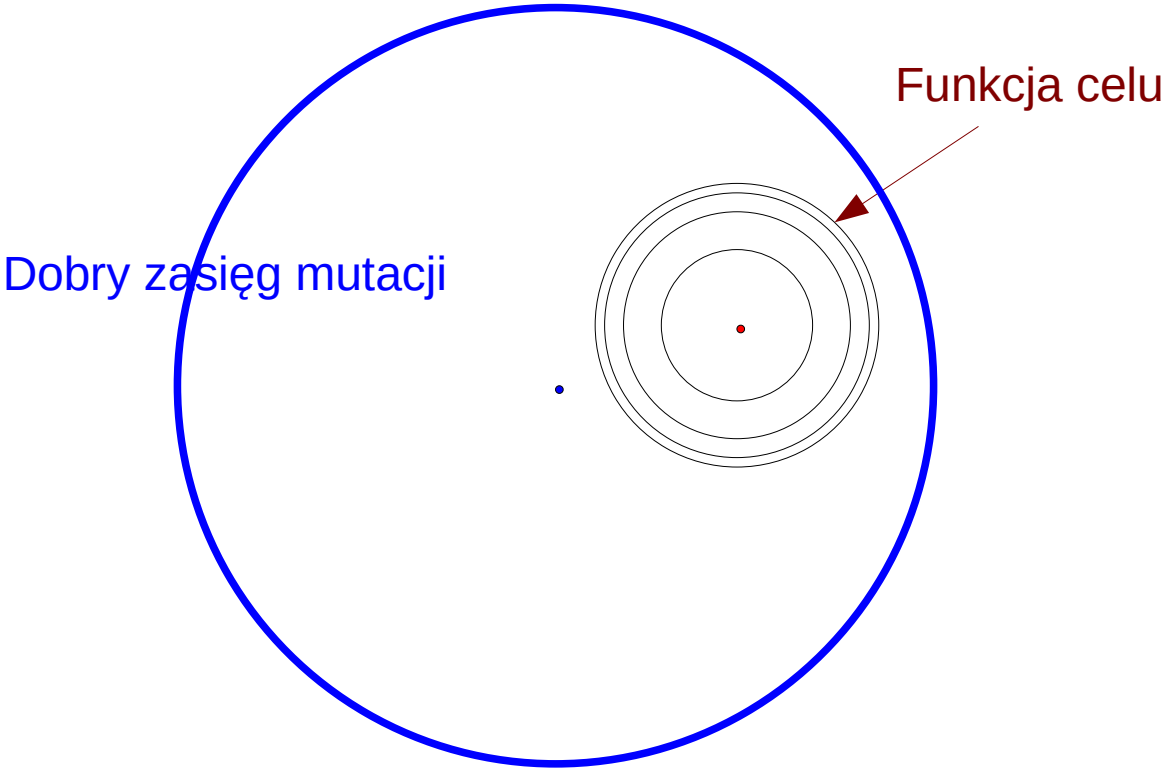
Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Zależność tempa zbieżności od zasięgu mutacji (alg. wspinaczkowy)

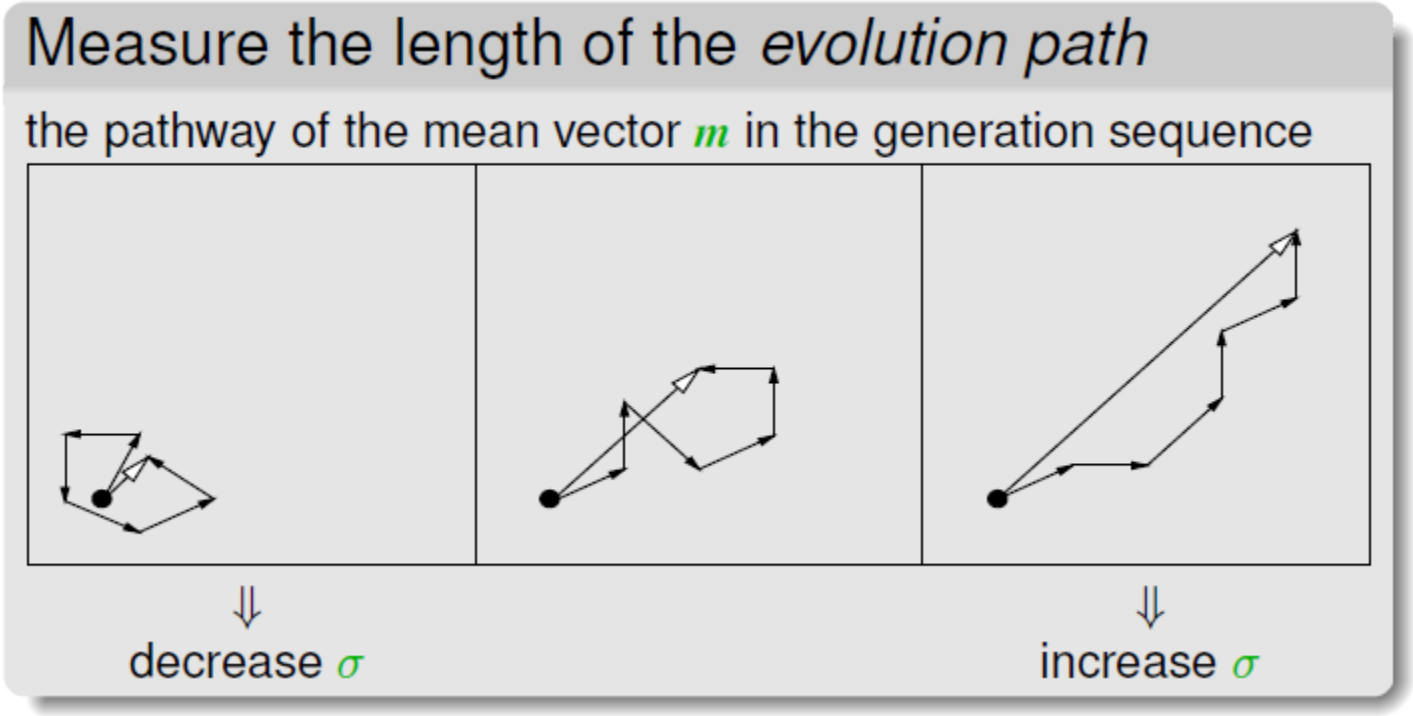


Zależność tempa zbieżności od zasięgu mutacji (alg. wspinaczkowy)

Zależność tempa zbieżności od zasięgu mutacji (alg. wspinaczkowy)



Adaptacja skumulowanego kroku algorytm CSA-ES



Trajektoria punktu środkowego
w przestrzeni wektorów
różnicowych

Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Adaptacja skumulowanego kroku algorytm CSA-ES

$$c_\sigma \approx 4/n, \quad d_\sigma \approx 1, \quad p_\sigma = 0$$

$$\textit{while ! stop}$$

$$\textit{generuj } d_i(t) \sim N(0, I), i = 1 \ldots \lambda$$

$$\textit{oblicz } q_i(t) = q\left(m(t) + \sigma(t) \cdot d_i(t)\right)$$

$$\textit{sortuj według } q_i(t)$$

$$\Delta(t) = \frac{1}{\mu} \sum_{i=1}^{\mu} d_i(t)$$

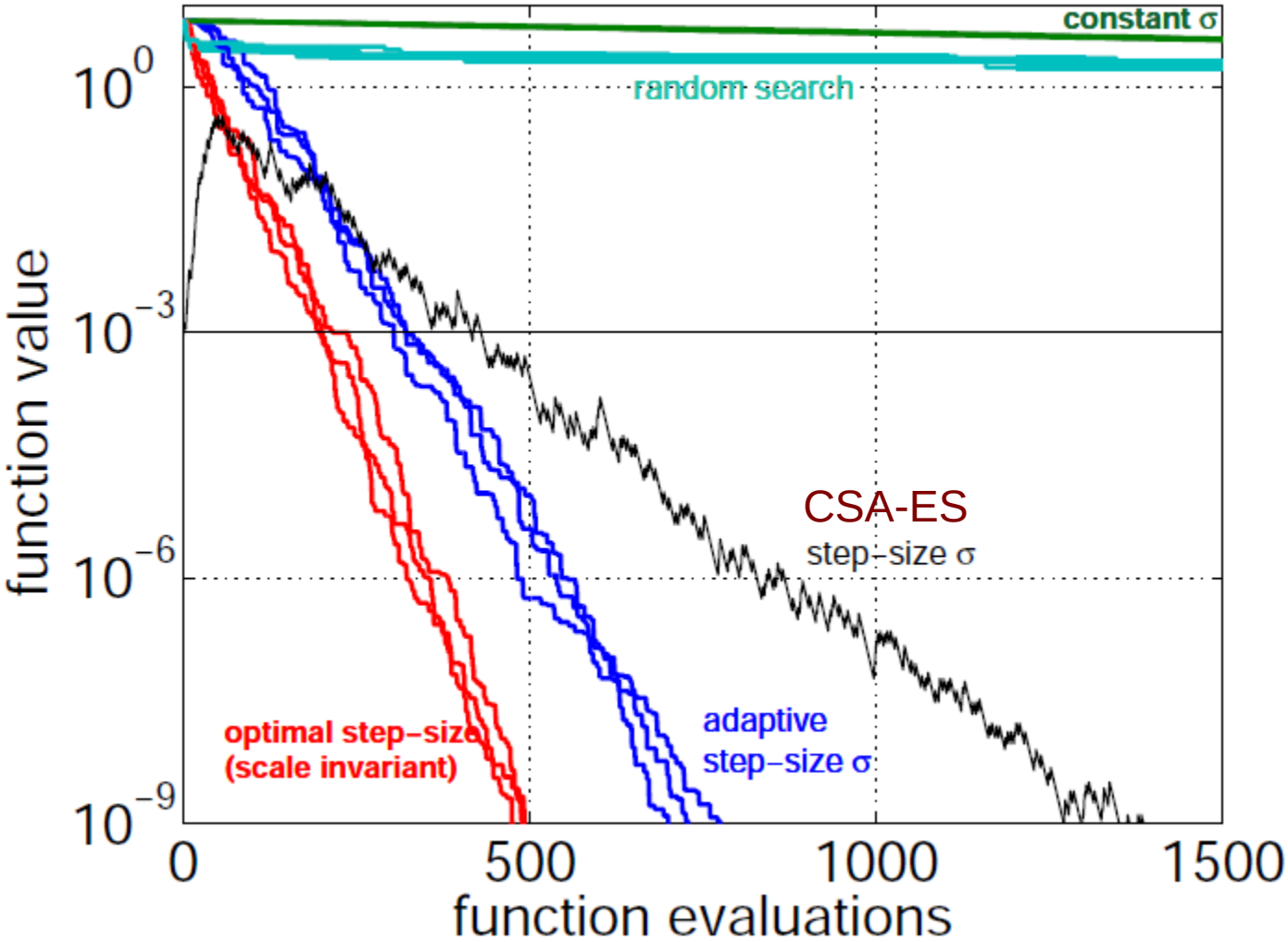
$$m(t+1) = m(t) + \sigma(t) \cdot \Delta(t)$$

$$p_\sigma(t+1) = (1 - c_\sigma)\, p_\sigma(t) + \sqrt{1 - (1 - c_\sigma)^2}\, \sqrt{\mu}\, \Delta(t)$$

$$\sigma(t+1) = \sigma(t) \cdot \exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|p_\sigma\|}{E\|N(0, I)\|} - 1\right)\right)$$

$$t \leftarrow t + 1$$

Zależność tempa zbieżności od zasięgu mutacji (alg. wspinaczkowy)



$$f(\boldsymbol{x}) = \sum_{i=1}^{n} x_i^2$$

in $[-0.2, 0.8]^n$
for $n = 10$

Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Modyfikacja macierzy kowariancji algorytm CMA-ES (wersja 0)

$$C(1)=I$$

$$\textit{while ! stop}$$

$$\textit{generuj } d_i(t) \sim N(0, C(t)), i=1 \ldots \lambda$$

$$\textit{oblicz } q_i(t)=q\left(m(t)+d_i(t)\right)$$

$$\textit{sortuj według } q_i(t)$$

$$\Delta(t)=\frac{1}{\mu}\sum_{i=1}^{\mu} d_i(t)$$

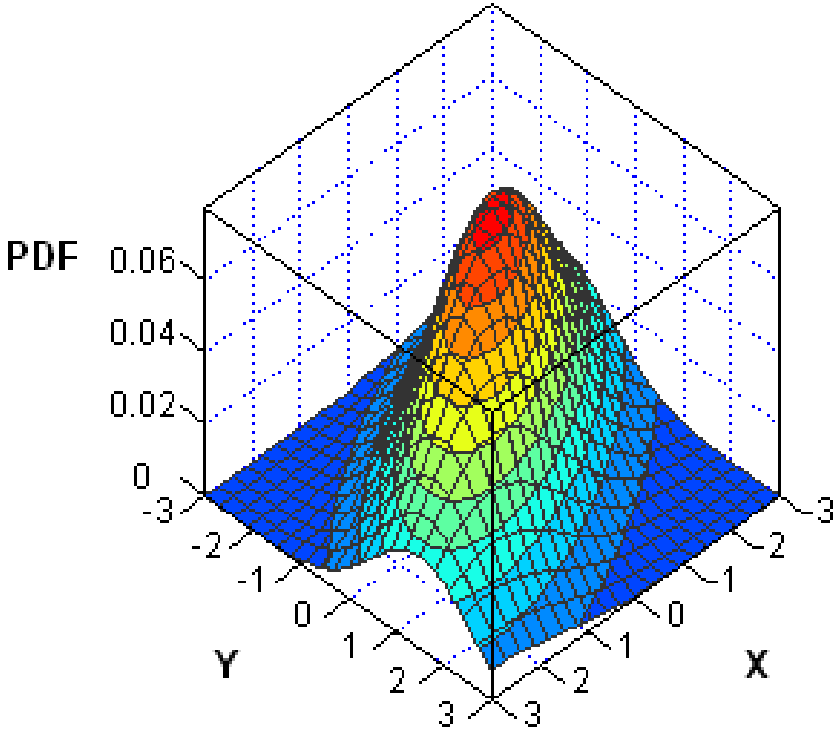
$$m(t+1)=m(t)+\Delta(t)$$

$$C(t+1)=(1-c_{cov})C(t)+c_{cov}\mu\,\Delta(t)\Delta(t)^T$$

$$t \leftarrow t+1$$

Jak wygenerować $d_i(t) \sim N(0, C(t))$

- Funkcja gęstości prawdopodobieństwa rozkładu normalnego z macierzą kowarancji C wektorem wartości oczekiwanej m

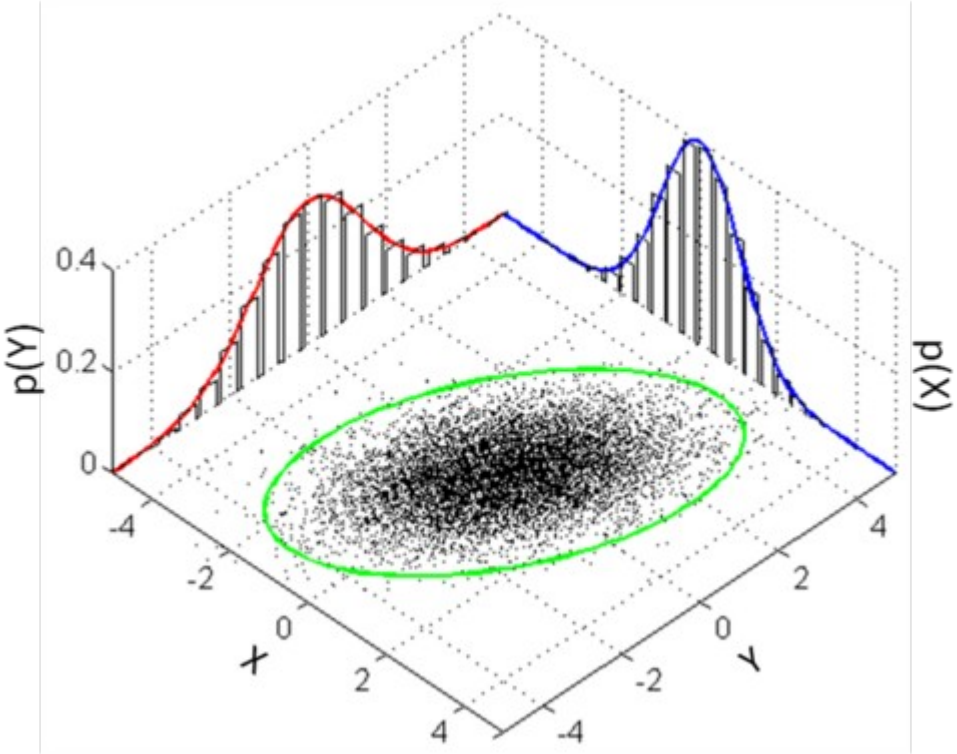


$$f(x) = (2\pi)^{-\frac{n}{2}} det(C)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x-m)^T C^{-1}(x-m)\right)$$

https://blogs.sas.com/content/iml/2012/07/05/compute-multivariate-normal-denstity.html

Jak wygenerować $d_i(t) \sim N(0, C(t))$

- Rozkład normalny ma poziomice gęstości będące elipsami

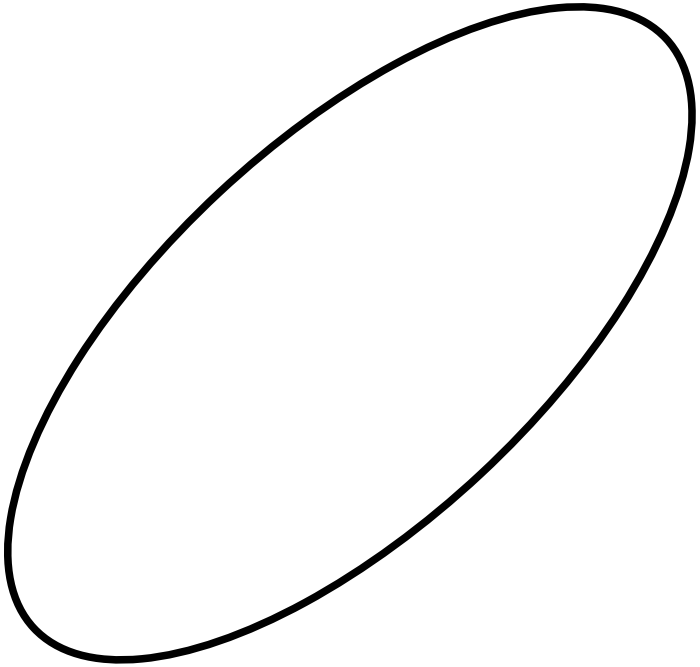


$$f(x) = (2\pi)^{-\frac{n}{2}} det(C)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x-m)^T C^{-1}(x-m)\right)$$

https://en.wikipedia.org/wiki/Multivariate_normal_distribution

Jak wygenerować $d_i(t) \sim N(0, C(t))$

- Rozkład normalny ma poziomice gęstości będące elipsami



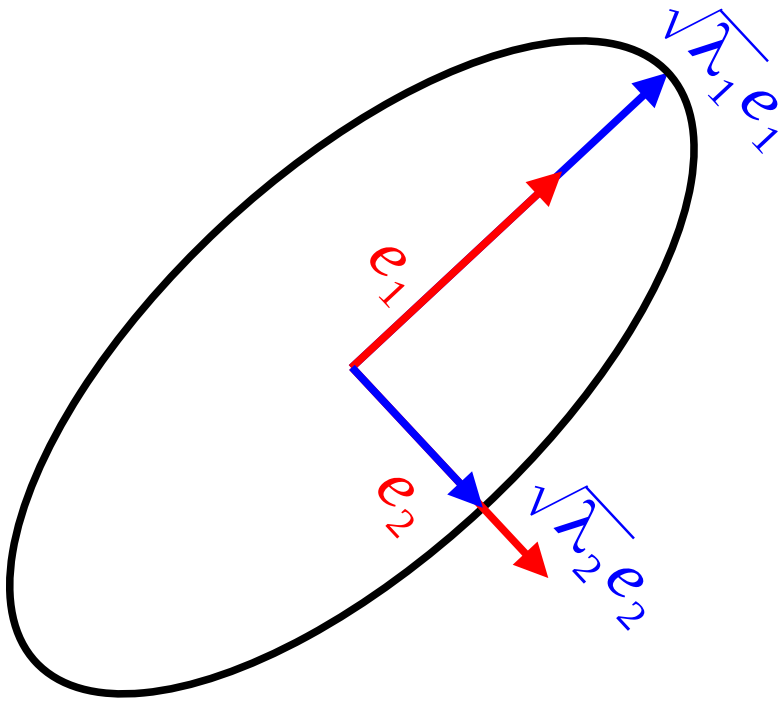
$$f(x) = (2\pi)^{-\frac{n}{2}} det(C)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x-m)^T C^{-1}(x-m)\right)$$

Jak wygenerować $d_i(t) \sim N(0, C(t))$

- Rozkład na wartości i wektory własne (eigenvalues, eigenvectors)

$$C = E \cdot \Lambda \cdot E^{-1}$$

- Wartości własne posortowane malejąco



Jak wygenerować $d_i(t) \sim N(0, C(t))$

- Niech $z_i(t) \sim N(0, I)$
- Rozkładamy macierz kowariancji

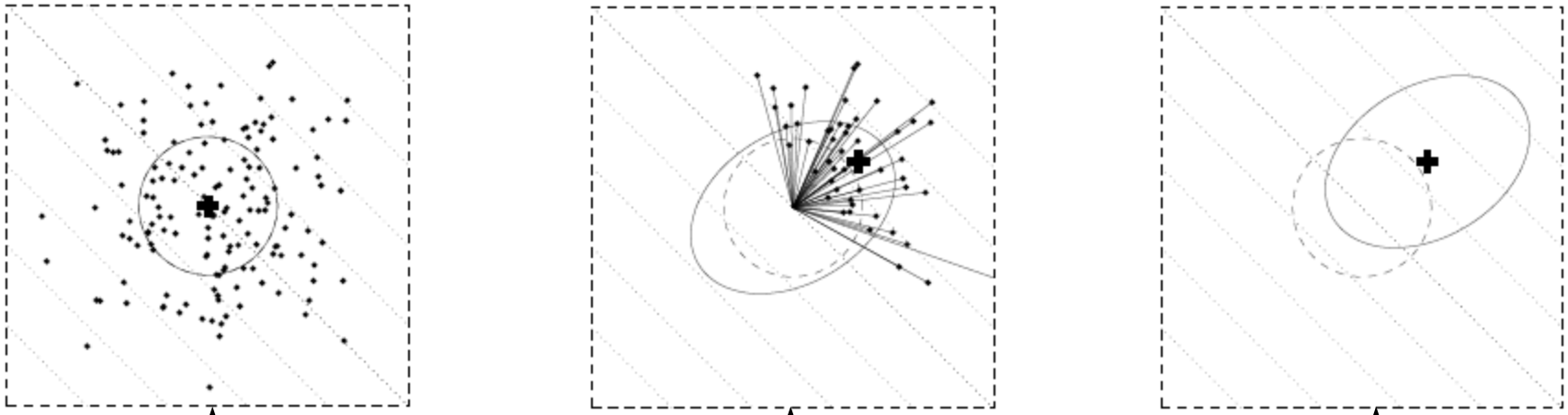
$$C(t) = E(t) \cdot \Lambda(t) \cdot E^{-1}(t)$$

- Ustalamy

$$d_i(t) = E(t) \Lambda^{\frac{1}{2}}(t) z_i(t)$$

- Gotowe!

Adaptacja macierzy kowariancji algorytm CMA-ES



$$\Delta(t) = \frac{1}{\mu} \sum_{i=1}^{\mu} d_i(t)$$

$$d_i(t) \sim \sigma(t) \cdot N(0, C(t)), i = 1 \ldots \lambda$$

$$C(t+1) = (1 - c_{cov}) C(t) + c_{cov} \mu \Delta(t) \Delta(t)^T$$

Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Modyfikacja macierzy kowariancji algorytm prawie CMA-ES (wersja 1)

$$C(1)=I$$

$$\textit{while ! stop}$$

$$\textit{generuj } d_i(t) \sim N(0, C(t)), i=1 \ldots \lambda$$

$$\textit{oblicz } q_i(t)=q\left(m(t)+d_i(t)\right)$$

$$\textit{sortuj według } q_i(t)$$

$$\Delta(t)=\frac{1}{\mu}\sum_{i=1}^{\mu} d_i(t)$$

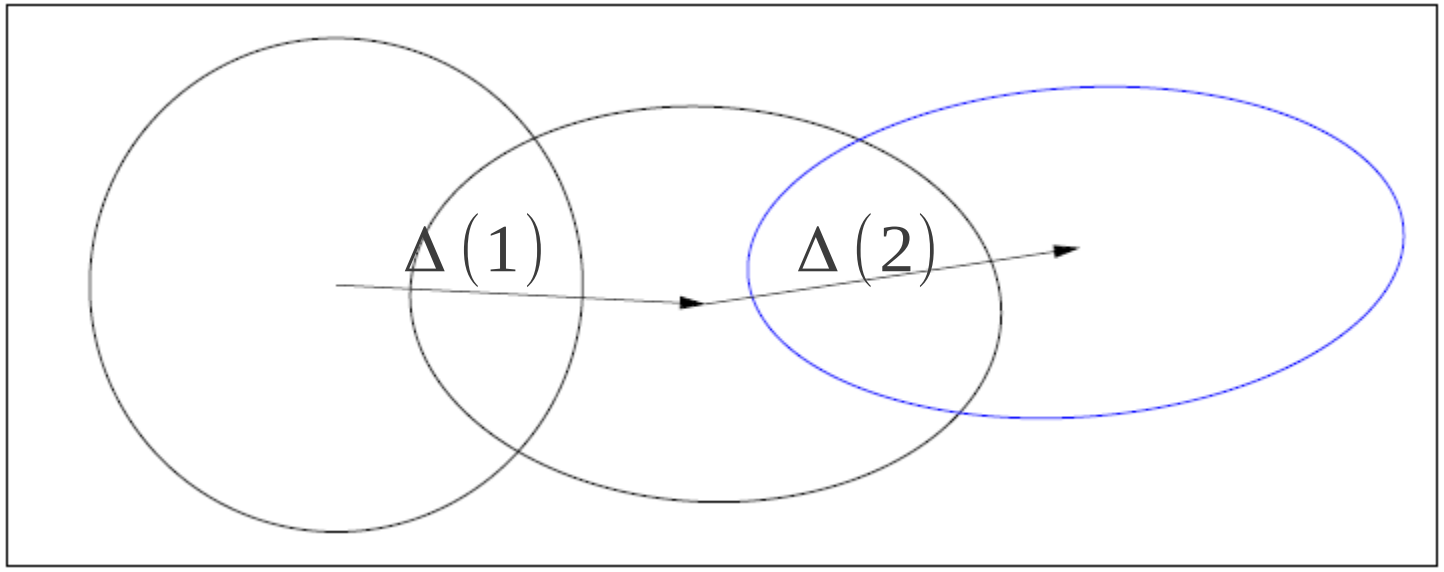
$$m(t+1)=m(t)+\Delta(t)$$

$$C(t+1)=(1-c_{cov})C(t)+c_{cov}\sum_{i=1}^{\mu} d_i(t)\,d_i(t)^T$$

$$t \leftarrow t+1$$

uwzględnienie wielu wektorów różnic

Adaptacja macierzy kowariancji algorytm CMA-ES



Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Modyfikacja macierzy kowariancji algorytm prawie CMA-ES (wersja 2)

$$C(1)=I, \quad p_c(1)=0$$

$$\textit{while ! stop}$$

$$\textit{generuj } d_i(t) \sim N(0, C(t)), i=1 \ldots \lambda$$

$$\textit{oblicz } q_i(t)=q(m(t)+d_i(t))$$

$$\textit{sortuj według } q_i(t)$$

$$\Delta(t)=\frac{1}{\mu}\sum_{i=1}^{\mu} d_i(t)$$

$$m(t+1)=m(t)+\Delta(t)$$

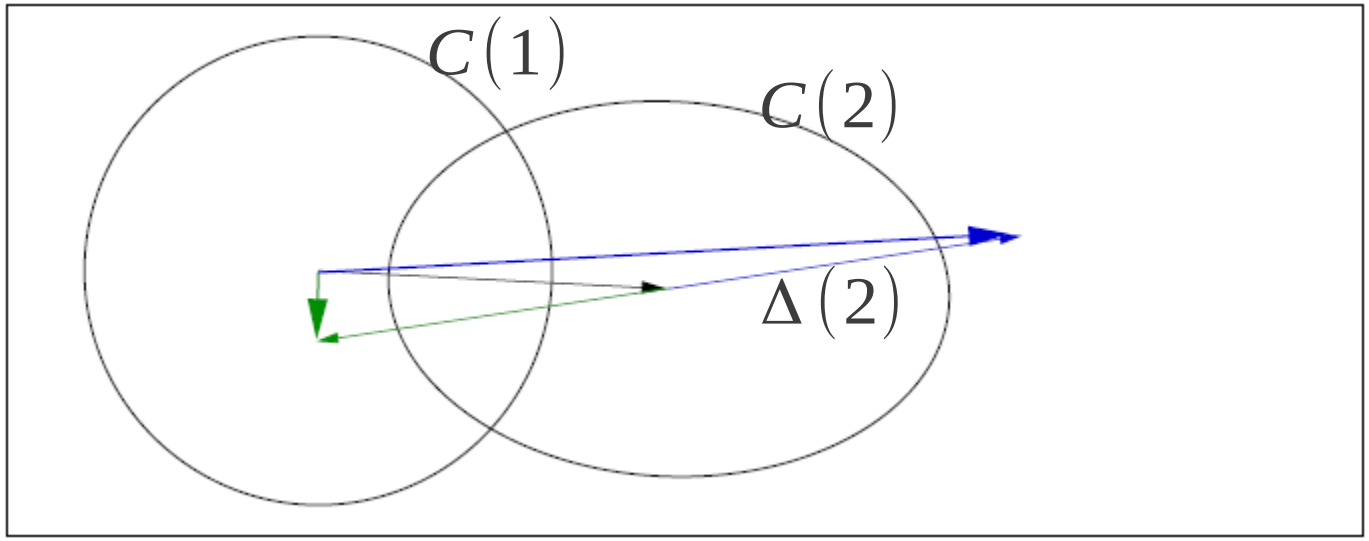
$$p_c(t+1)=(1-c_c)\,p_c(t)+\sqrt{1-(1-c_c)^2}\,\sqrt{\mu}\,\Delta(t)$$

bezwładność

$$C(t+1)=(1-c_{cov})\,C(t)+c_{cov}\,p_c(t+1)\,p_c(t+1)^T$$

$$t \leftarrow t+1$$

Adaptacja skumulowanego kroku



$$p_c(t+1)=(1-c_c)\,p_c(t)+\sqrt{1-(1-c_c)^2}\,\sqrt{\mu}\,\Delta(t)$$
$$C(t+1)=(1-c_{cov})\,C(t)+c_{cov}\,p_c(t+1)\,p_c(t+1)^T$$

Rysunek z: *A. Auger, N. Hansen:*
CMA-ES — Evolution Strategies and Covariance Matrix Adaptation, tutorial, GECCO'2011

Pełny CMA-ES

$$C(1)=I, \quad p_c(1)=0, \quad p_\sigma(1)=0, \quad \mu_{eff}=1/\sum_{i=1\ldots\mu}(w_i)^2$$

$$\begin{aligned}
&\textit{while ! stop}\\
&\quad \textit{generuj } d_i(t)\sim N(0,C(t)), i=1\ldots\lambda\\
&\quad \textit{oblicz } q_i(t)=q\big(m(t)+\sigma(t)\cdot d_i(t)\big)\\
&\quad \textit{sortuj według } q_i(t)\\
&\quad \Delta(t)=\sum_{i=1\ldots\mu} w_i d_i(t)\\
&\quad m(t+1)=m(t)+\sigma(t)\cdot\Delta(t)\\
&\quad p_\sigma(t+1)=(1-c_\sigma)p_\sigma(t)+C^{-1/2}\sqrt{1-(1-c_\sigma)^2}\sqrt{\mu_{eff}}\,\Delta(t)\\
&\quad p_c(t+1)=(1-c_c)p_c(t)+\sqrt{1-(1-c_c)^2}\sqrt{\mu_{eff}}\,\Delta(t)\\
&\quad \sigma(t+1)=\sigma(t)\cdot\exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|p_\sigma\|}{E\|N(0,I)\|}-1\right)\right)\\
&\quad C(t+1)=(1-c_1-c_\mu)C(t)+c_1 p_c(t+1)p_c(t+1)^T+c_\mu\sum_{i=1\ldots\mu} w_i d_i(t)d_i(t)^T\\
&\quad t\leftarrow t+1
\end{aligned}$$

Beyer, Sendhoff, Simplify Your Covariance Matrix AdaptationEvolution Strategy, IEEE Trans on Evol. Comp. 2017

BFGS

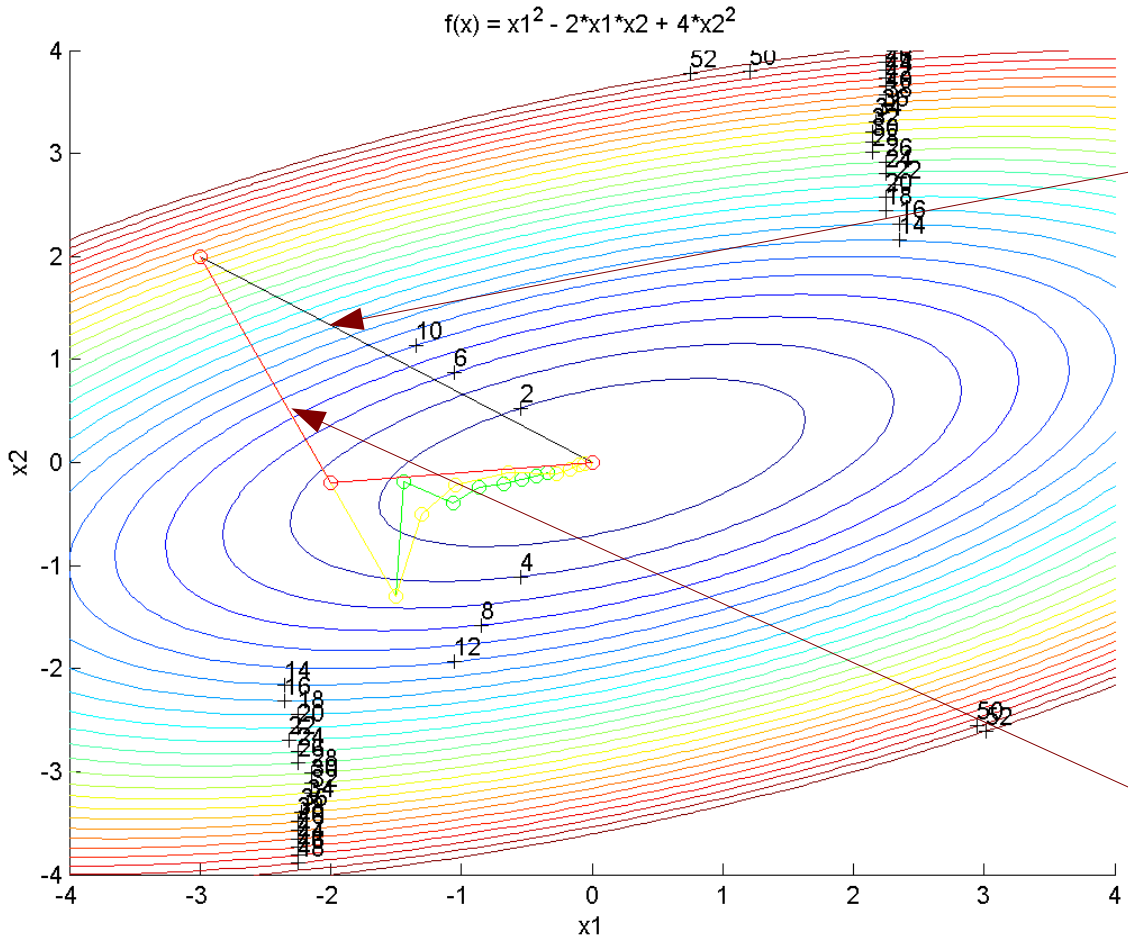
1. Obtain a direction $\mathbf{p}_k$ by solving $\mathbf{p}_k = -H_k \nabla f(\mathbf{x}_k)$.
2. Perform a one-dimensional optimization (line search) to find an acceptable stepsize α_k in the direction found in the first step. If an exact line search is performed, then $\alpha_k = \arg\min f(\mathbf{x}_k + \alpha \mathbf{p}_k)$. In practice, an inexact line search usually suffices, with an acceptable α_k satisfying Wolfe conditions.
3. Set $\mathbf{s}_k = \alpha_k \mathbf{p}_k$ and update $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k$.
4. $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$.
5. $H_{k+1} = H_k + \frac{(\mathbf{s}_k^{\mathrm{T}} \mathbf{y}_k + \mathbf{y}_k^{\mathrm{T}} H_k \mathbf{y}_k)(\mathbf{s}_k \mathbf{s}_k^{\mathrm{T}})}{(\mathbf{s}_k^{\mathrm{T}} \mathbf{y}_k)^2} - \frac{H_k \mathbf{y}_k \mathbf{s}_k^{\mathrm{T}} + \mathbf{s}_k \mathbf{y}_k^{\mathrm{T}} H_k}{\mathbf{s}_k^{\mathrm{T}} \mathbf{y}_k}$.

H_k oznacza aproksymację odwrotności hesjanu

typowo $H_1 = I$

https://en.wikipedia.org/wiki/Broyden%E2%80%93Fletcher%E2%80%93Goldfarb%E2%80%93Shanno_algorithm

BFGS

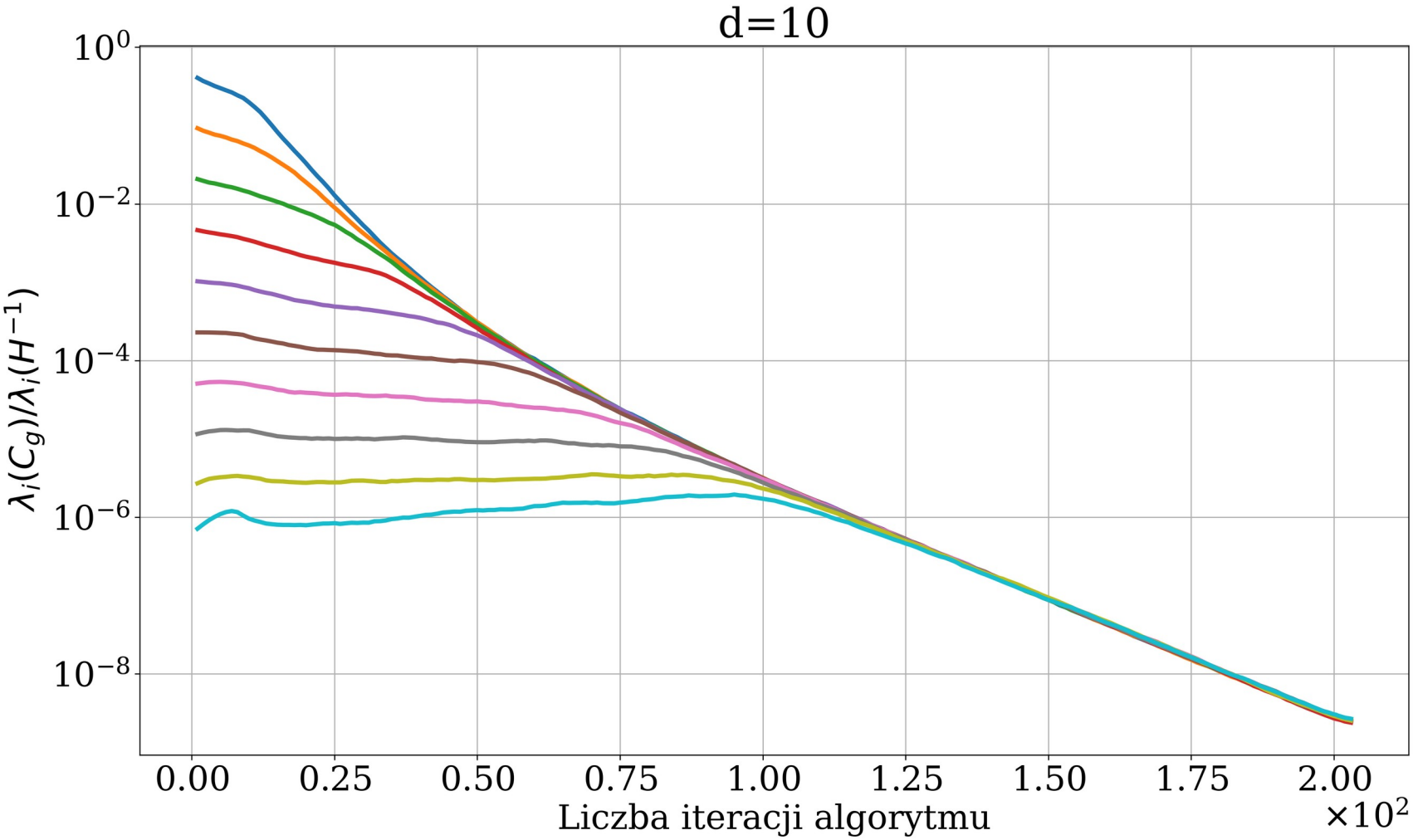


$$-H\nabla q(x)$$

$$-I\nabla q(x)$$

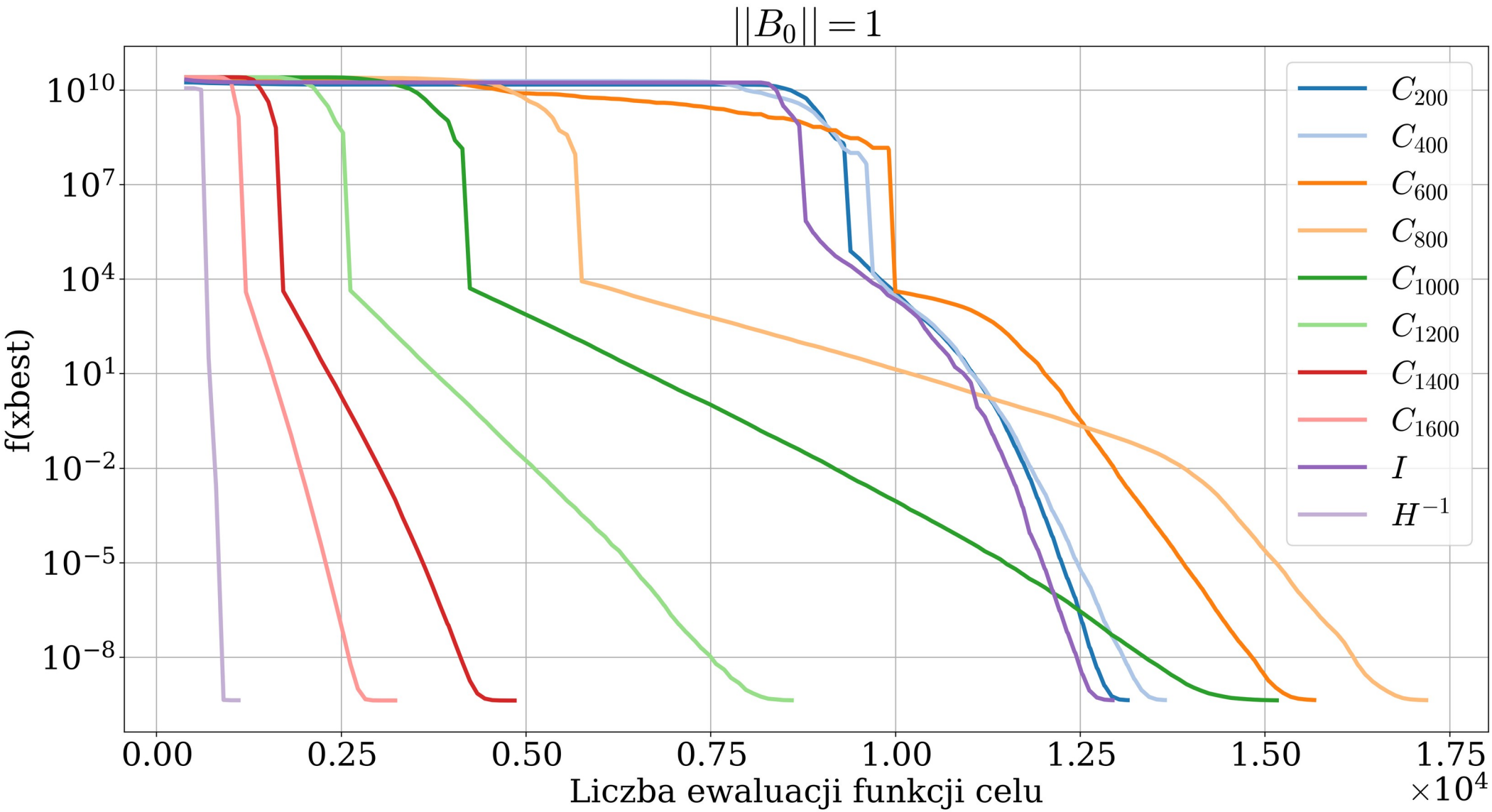
https://apmonitor.com/me575/index.php/Main/QuasiNewton

CMA-ES – adaptacja macierzy kowariancji a geometria funkcji celu



Dla funkcji kwadratowej z czasem C -> H^{-1}

„Turbodoładowany” BFGS



Algorytm **CMABFGS**

parametry wejściowe: k, x_0

$g \leftarrow 0$

dopóki nie spełniono kryterium zatrzymania:

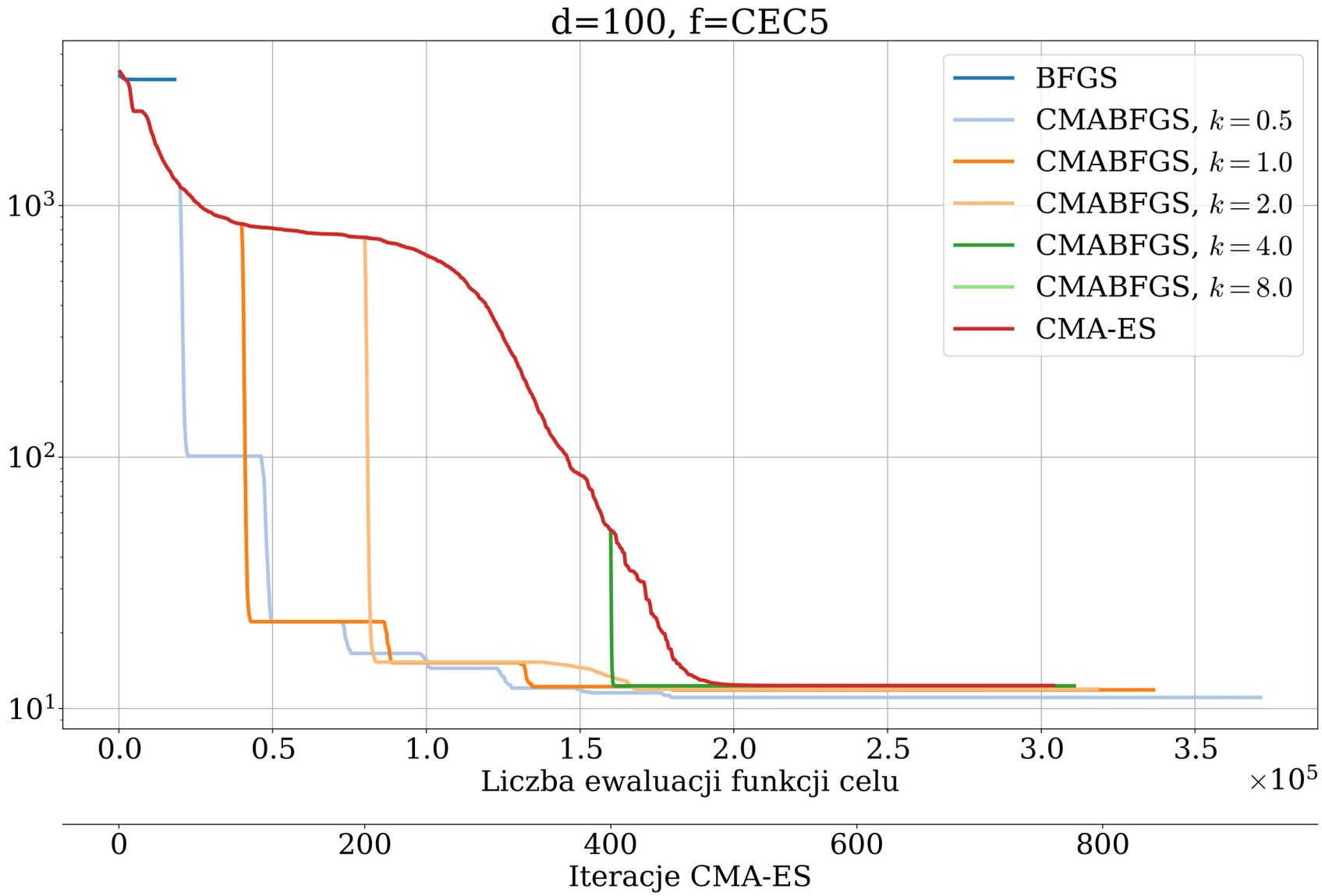
  $g \leftarrow g + 1$

  $\boldsymbol{m}_g, \boldsymbol{C}_g \leftarrow$ iter_cma_es()

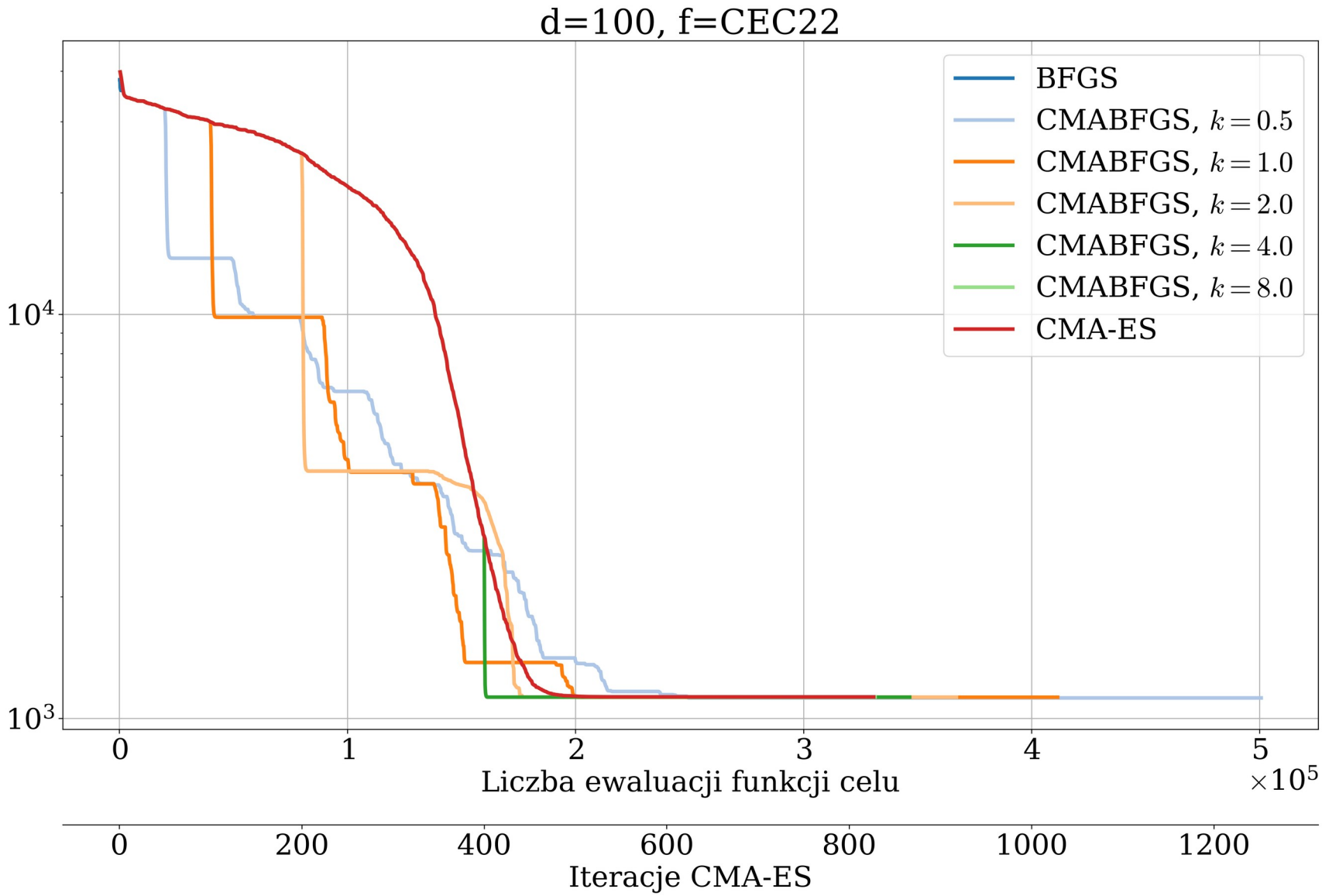
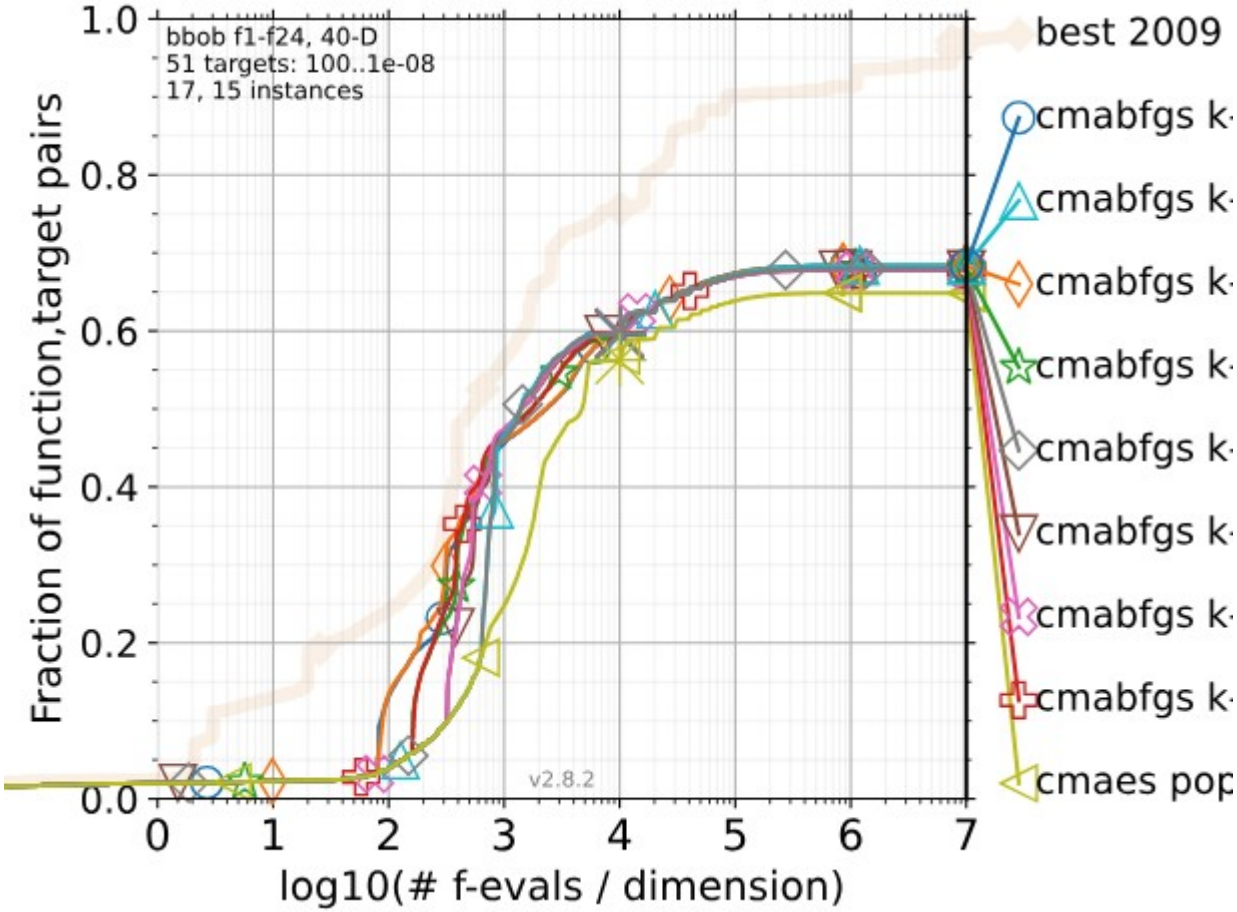
  jeżeli $g\%(k \cdot d) == 0$:

    BFGS($\boldsymbol{x}_0$=$\boldsymbol{m}_g$, $\boldsymbol{B}_0$=normalizacja($\boldsymbol{C}_g$))

CMABFGS – benchmark CEC



CMABFGS – benchmark CEC



CMABFGS – benchmark BBOB